



Mixed-Type Data Quantification

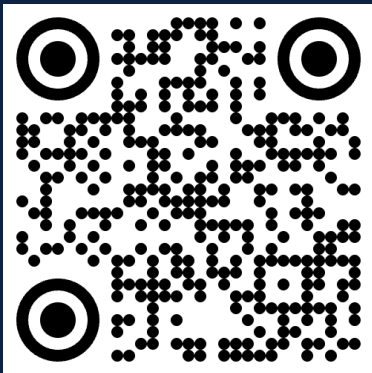
The Advanced R Series: Clustering & Classification

Slides and code created by Jesse Ghashti.

March 4, 2026

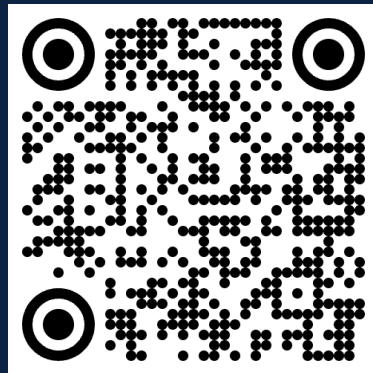
Centre for Scholarly Communication

The University of British Columbia | Okanagan Campus | Syilx Okanagan Nation Territory



←
Check out our other
CSC workshops!

GitHub code and
slides for today's
workshop (and pre-
vious workshops)



Alternatively, code/slides available at the bottom of
<https://csc-ubc-okanagan.github.io/workshops/>

Workshop Series Overview



Session	Topic	Date/Time
1	Hierarchical Models	Jan 14, 12:00 PM
2	Centroid-Based Models	Jan 19, 3:30 PM
3	Fuzzy Clustering	Jan 28, 1:00 PM
4	Distribution-Based Models	Feb 2, 3:30 PM
5	Density-Based Models	Feb 9, 3:30 PM
6	Graph-Based Models	Feb 23, 3:30 PM
7	Mixed-Type Data Quantification	Mar 4, 1:00 PM
8	Mixed-Type Data Clustering	Mar 11, 1:00 PM
9	Bias Reduction and Fairness	Mar 18, 1:00 PM
10	Dimensionality Reduction 1 of 2	Mar 23, 3:30 PM
11	Dimensionality Reduction 2 of 2	Mar 30, 3:30 PM

What we learned...

- Constructed similarity graphs from coordinate data
- Learned the graph Laplacian and its spectral properties
- Used eigendecomposition and the eigengap for cluster selection
- Applied spectral clustering to non-convex cluster shapes
- Introduced modularity-based community detection (Louvain)
- Compared methods on infrastructure and airport networks

Today: Mixed-Type Data Quantification



Today we will...

- Understand why real data is often mixed-type
- Explore naive approaches (scaling, dummy coding) and their limitations
- Learn Gower distance for mixed-type data
- Compare alternative measures: Podani, Wishart, Huang
- Introduce variable weighting for domain knowledge
- Examine geographic distances (Haversine vs Euclidean)

Today we require...

```
library('cluster')  
library('kmed')  
library('ggplot2')  
library('dplyr')  
library('maps')  
library('ggrepel')
```

The Problem: Real Data is Mixed-Type



Variable Types in Real Datasets

So far, we have clustered continuous (numeric) data.

But real datasets often contain

- **Continuous:** age, income, temperature
- **Categorical/Nominal:** colour, country, diagnosis
- **Ordinal:** education level, satisfaction rating
- **Binary:** yes/no, true/false
- **etc.**

How do we compute distances when variables are on different scales and types?

A Metric in a Metric Space

A **distance function** $d : X \times X \rightarrow \mathbb{R}$ is a **metric** if it satisfies:

1. **Non-negativity:** $d(x, y) \geq 0$
2. **Identity:** $d(x, y) = 0 \iff x = y$
3. **Symmetry:** $d(x, y) = d(y, x)$
4. **Triangle inequality:** $d(x, z) \leq d(x, y) + d(y, z)$

Dissimilarity $\neq$ Distance A dissimilarity only requires non-negativity and symmetry. Many clustering algorithms use dissimilarities that violate metric properties.

Term	Violated Property
Semimetric / Premetric	Triangle inequality
Pseudometric	Identity (distinct points can have $d = 0$)
Quasimetric	Symmetry ($d(x, y) \neq d(y, x)$)

From Distance d to Similarity s

The conversion depends on the **range** of your distance measure

For bounded distances $d \in [0, 1]$, you could use $s = 1 - d$

For unbounded distances $d \in [0, \infty)$ (e.g., Euclidean):

$$s = \frac{1}{1 + d} \quad \text{or} \quad s = e^{-d} \quad \text{or} \quad s = e^{-d^2/2\sigma^2}$$

The last is the Gaussian/RBF kernel (we used this in spectral clustering).

Similarity has weaker properties... Similarities are not metrics. They don't satisfy the triangle inequality, and the "identity" property becomes $s(x, x) = 1$ (maximum similarity). Many theoretical guarantees for distance-based algorithms don't transfer directly to similarity-based versions.

We won't distinguish between distance metrics or dissimilarities throughout the rest of this talk. But, it's important to note the difference. For example, the commonly used squared Euclidean distance

$$d(\mathbf{x}, \mathbf{y}) = (\mathbf{x} - \mathbf{y})^\top (\mathbf{x} - \mathbf{y})$$

is not a distance metric (does not satisfy the triangle inequality), whereas

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top (\mathbf{x} - \mathbf{y})}$$

is a distance metric.

Later... Gower distance is a metric, whereas Podani and Huang are not (these names will make sense soon)

Simulated Mixed-Type Dataset



```
n <- 150
mixedDf <- data.frame(
  age = round(rnorm(n,45,15)),
  income = round(rlnorm(n,10.5,0.8)),
  yearsExperience = pmax(0, round(rnorm(n,15,8))),
  education = factor(sample(c("HighSchool", "Bachelors", "Masters", "PhD"), n,
    replace=TRUE, prob=c(0.3,0.4,0.2,0.1)),
    levels=c("HighSchool", "Bachelors", "Masters", "PhD"), ordered=TRUE),
  region = factor(sample(c("North", "South", "East", "West"), n, replace=TRUE)),
  employed = sample(c(TRUE, FALSE), n, replace=TRUE, prob=c(0.7,0.3)),
  satisfaction = factor(sample(1:5, n, replace=TRUE), levels=1:5, ordered=TRUE),
  sector = factor(sample(c("Tech", "Finance", "Healthcare", "Retail", "Education"), n, replace=TRUE)))
```

age	income	yearsExperience	education	region	employed	satisfaction	sector
61	101891	19	HighSchool	North	FALSE	1	Retail
36	11185	24	HighSchool	West	FALSE	2	Finance
53	54730	9	HighSchool	North	TRUE	3	Education
63	133217	18	HighSchool	East	TRUE	5	Finance
30	25832	13	Bachelors	North	TRUE	3	Healthcare
34	22122	16	PhD	South	TRUE	5	Education

Common “Solutions”

Approach 1: Treat everything as numeric (convert factors to integers)

Approach 2: Z-score standardization (mean=0, sd=1)

Approach 3: Min-max scaling (scale to $[0, 1]$)

Approach 4: Dummy/one-hot encoding (binary indicators)

Each approach has potentially serious limitations (especially in the context of clustering and classification) that we will explore.

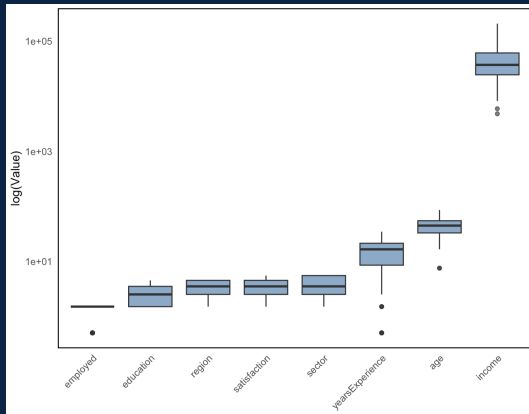
Approach 1: Treat Everything as Numeric



```
naiveNumeric <- mixedDf %>%  
  mutate(  
    education = as.numeric(education),  
    region = as.numeric(region),  
    employed = as.numeric(employed),  
    satisfaction = as.numeric(satisfaction),  
    sector = as.numeric(sector)) %>%  
  select(-trueCluster)  
  
distRaw <- dist(naiveNumeric, method="euclidean")
```

The Scale Problem

Income ranges in thousands, age in tens, factors in 1–5. Income will completely dominate the distance calculation.



Approach 2: Z-Score Standardization

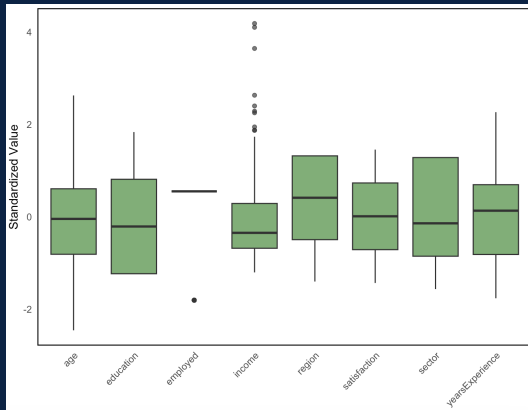


```
zScoreDf <- naiveNumeric %>%  
  mutate(across(everything(),  
    ~(. - mean(.)) / sd(.)))  
  
round(colMeans(zScoreDf), 10)  
round(apply(zScoreDf, 2, sd), 10)  
distZscore <- dist(zScoreDf, method="euclidean")
```

Z-score formula

$$z_i = \frac{x_i - \bar{x}}{s_x}$$

All variables now have mean = 0 and
sd = 1.



Approach 3: Min-Max Scaling

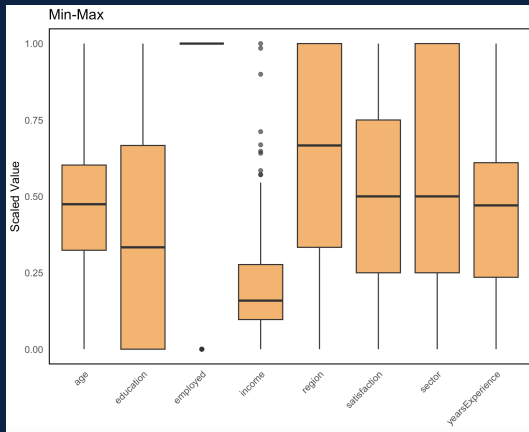


```
minMaxDf <- naiveNumeric %>%  
  mutate(across(everything(),  
    ~(. - min()) / (max() - min())))  
distMinMax <- dist(minMaxDf, method="euclidean")
```

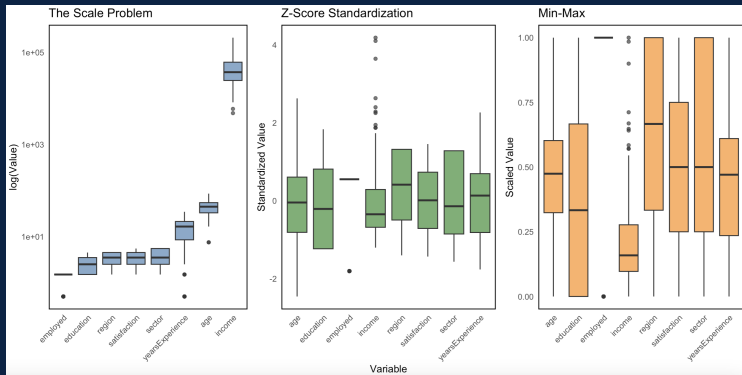
Min-max formula

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

All variables now in $[0, 1]$.



Comparing the Three Approaches



Z-score and min-max address the scale problem, but they still treat categorical variables as numeric.

Approach 4: Dummy/One-Hot Encoding



```
dummyDf <- model.matrix(~.-1,  
  data = mixedDf %>% select(-trueCluster))
```

We went from 8 variables to many more! Each category becomes its own binary column.

educationHighSchool	educationBachelors	educationMasters	educationPhD
1	0	0	0
1	0	0	0
0	0	1	0
0	1	0	0
1	0	0	0
0	1	0	0

Limitation 1: Ordinal Information Lost

With dummy coding, HighSchool < Bachelors < Masters < PhD. But

$$d(\text{HighSchool}, \text{Bachelors}) = d(\text{HighSchool}, \text{PhD}) = \sqrt{2}$$

PhD should be *further* from HighSchool than Bachelors is?

Limitation 2: Artificial Ordering for Nominal

Converting region to numeric: North=1, South=2, East=3, West=4 (for example).

This implies West > East > South > North, which is *meaningless*.

$$d(\text{North}, \text{South}) = 1, \text{ but } d(\text{North}, \text{West}) = 3?$$

Limitation 3: Binary Variables May Need Asymmetric Treatment

employed: TRUE/FALSE

If clustering job seekers, joint unemployment might be informative. If clustering general population, it might not be.

Should we treat $(0, 0)$ the same as $(1, 1)$?

Limitation 4: Distributional Assumptions

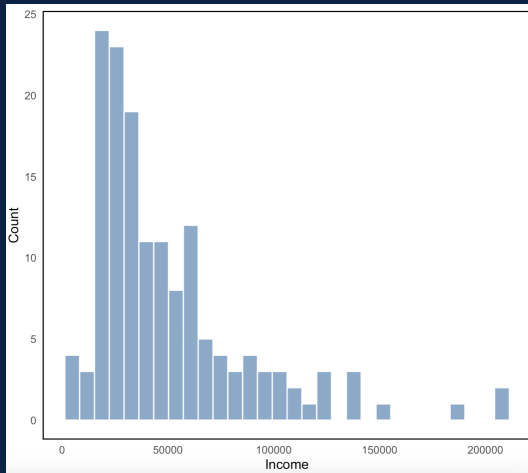
Z-score assumes approximate normality. Highly skewed variables (like income) violate this assumption. Outliers heavily affect both z-score and min-max scaling.

Limitation 4: Skewed Distributions

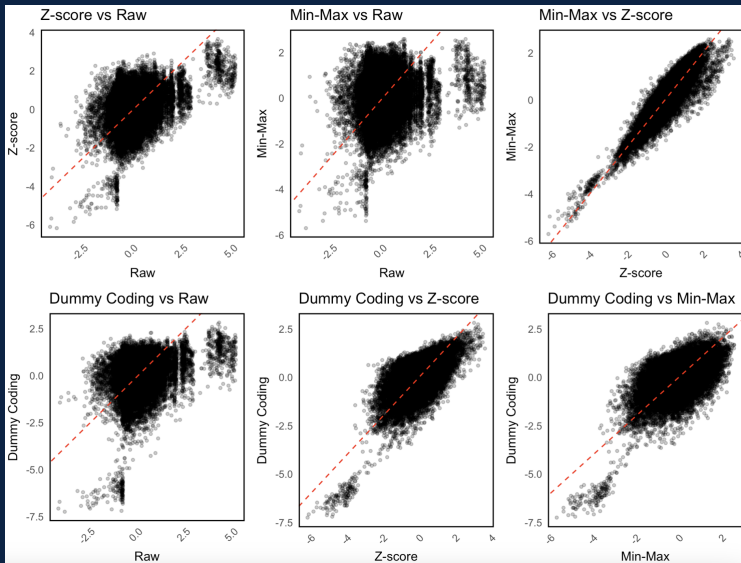


```
incomeSkew <- round(  
  moments::skewness(mixedDf$income), 2)
```

Income is highly right-skewed. Z-score standardization assumes symmetry around the mean. Consider log-transformation before scaling.



Do These Distances Agree?



If two distance measures capture the same structure, points should lie along the diagonal. Deviations indicate the measures disagree on relative distances.

A Proper Mixed-Type Distance

Gower distance combines type-specific components. For a variable k , treat it based on variable type as one of the following:

- **Continuous:** $d_k = \frac{|x_{ik} - x_{jk}|}{\text{range}_k}$
- **Ordinal:** Same as continuous, applied to ranks
- **Nominal:** $d_k = 0$ if same, 1 if different
- **Binary:** $d_k = 0$ if same, 1 if different

Final distance is the weighted average of components

$$d_{ij} = \frac{\sum_k w_k \cdot d_{ijk}}{\sum_k w_k \cdot \delta_{ijk}}$$

where $\delta_{ijk} = 1$ if comparison is valid, 0 if missing, and w_k is a weight applied to each variable based on their importance (you can set to 1 for now, and we will discuss this shortly).

```
mixedForDist <- mixedDf %>% select(-trueCluster)
gowerDist <- daisy(mixedForDist, metric="gower")
summary(gowerDist)
```

`daisy()` from the `cluster` package automatically detects variable types:

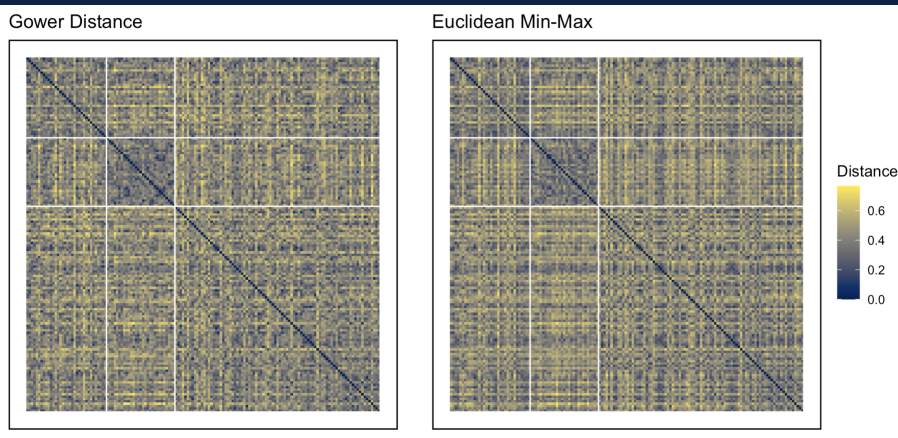
- Numeric → continuous
- Ordered factor → ordinal
- Unordered factor → nominal
- Logical → binary

```
11175 dissimilarities, summarized :
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.04069 0.34415 0.42370 0.42179 0.50244 0.76657
Metric : mixed ; Types = I, I, I, O, N, A, O, N
Number of objects : 150
```

Compare the manual calculation below to the one by `daisy`. Below matches the formula we gave for Gower distance.

```
obs1 <- mixedForDist[1,]
obs2 <- mixedForDist[2,]
# Continuous
dAge <- abs(obs1$age - obs2$age) / (max(mixedForDist$age) - min(mixedForDist$age))
dIncome <- abs(obs1$income - obs2$income) / (max(mixedForDist$income) - min(mixedForDist$income))
# Ordinal
dEducation <- abs(as.numeric(obs1$education) - as.numeric(obs2$education)) /
  (length(levels(mixedForDist$education)) - 1)
# Nominal
dRegion <- as.numeric(obs1$region != obs2$region)
# Binary
dEmployed <- as.numeric(obs1$employed != obs2$employed)
# Final Gower
manualGower <- mean(c(dAge, dIncome, dExp, dEducation, dRegion, dEmployed, dSatisfaction, dSector))
```

Visualizing Mixed-Type Distance Matrices



Rows/columns sorted by true cluster. Darker blocks along diagonal = lower within-cluster distances. Gower captures mixed-type structure differently than Euclidean.

The `kmed` Package: `distmix()`

Multiple methods for mixed-type data:

- **Gower**: Standard Gower distance
- **Podani**: Better ordinal handling (rank differences)
- **Wishart**: Alternative continuous handling
- **Huang**: Used in k -prototypes (next workshop!)

Requires specifying column indices for each type:

- `idnum`: numeric columns
- `idbin`: binary columns
- `idcat`: categorical columns

Alternative, use `?kmed::distmix` in the console, check the formulas, and compute manually as an exercise.

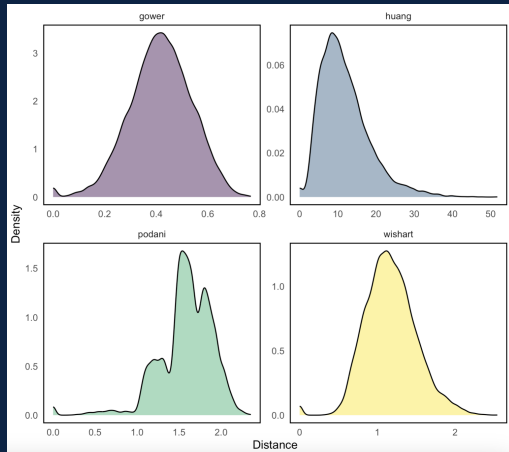
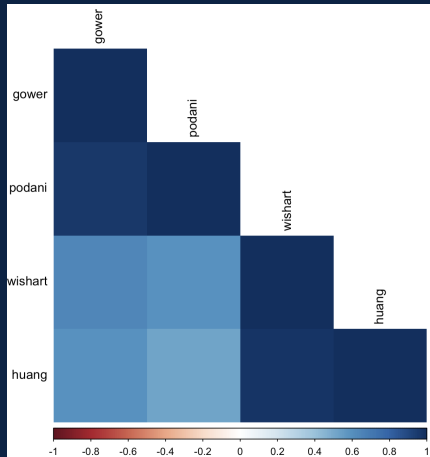
Using distmix()



```
mixedMatrix <- mixedForDist %>%  
  mutate(education=as.numeric(education), satisfaction=as.numeric(satisfaction),  
         region=as.numeric(region), sector=as.numeric(sector),  
         employed=as.numeric(employed)) %>%  
  as.data.frame()  
  
idnum <- c(1,2,3,4,7)  
idbin <- c(6)  
idcat <- c(5,8)  
  
distGowerKmed <- distmix(mixedMatrix, method="gower", idnum=idnum, idbin=idbin, idcat=idcat)  
distPodani <- distmix(mixedMatrix, method="podani", idnum=idnum, idbin=idbin, idcat=idcat)  
distWishart <- distmix(mixedMatrix, method="wishart", idnum=idnum, idbin=idbin, idcat=idcat)  
distHuang <- distmix(mixedMatrixSc, method="huang", idnum=idnum, idbin=idbin, idcat=idcat)
```

Note: Huang method requires scaled numeric variables

Correlation Between Methods

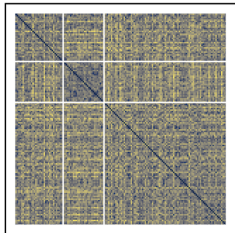


Methods are correlated but not identical. Gower and Podani are most similar. Huang has a different distribution.

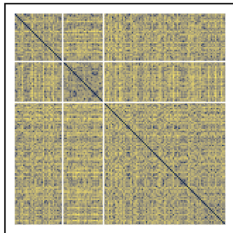
Distance Matrices by Method



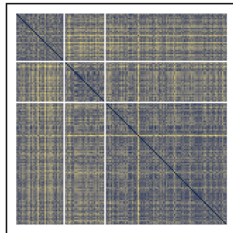
Gower



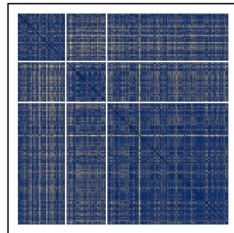
Podani



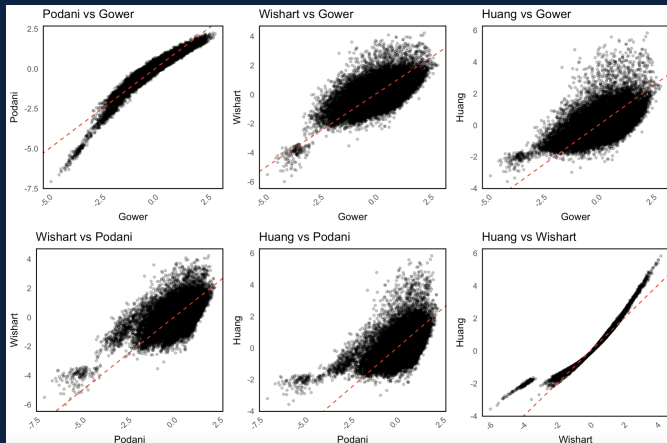
Wishart



Huang



Pairwise Method Comparisons



Points along the diagonal indicate agreement. Spread indicates the methods weight variables differently.

Incorporating Domain Knowledge

Not all variables are equally important!

Example In a hiring context, income and education might matter more than region or sector.

Gower distance allows variable weights:

$$d_{ij} = \frac{\sum_k w_k \cdot d_{ijk}}{\sum_k w_k}$$

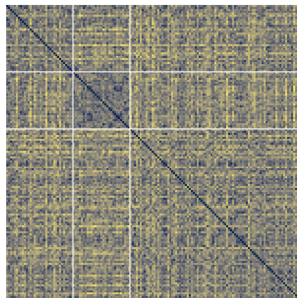
Higher weight $\rightarrow$ more influence on distance.

Weighted Gower in R

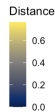
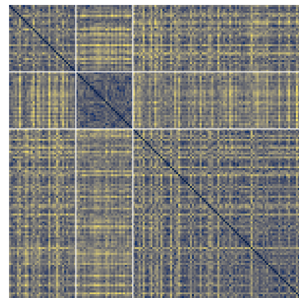


```
weightsImportant <- c(  
  age = 0.5,  
  income = 2,  
  yearsExperience = 0.5,  
  education = 2,  
  region = 0.5,  
  employed = 1,  
  satisfaction = 1,  
  sector = 0.5)  
  
gowerWeighted <- daisy(mixedForDist,  
  metric = "gower",  
  weights = weightsImportant)
```

Unweighted



Weighted



Weighting changes the distance structure. Pairs that differ on income/education move further apart; pairs that differ on region/sector move closer together.

How to Choose Weights?



This an entire field of statistics! So, we don't have enough time to dip our toes in here, but the following are a couple starting approaches to consider...

1. **Domain expertise** Which variables matter most for distinguishing meaningful groups, then weight accordingly
2. **Equal weights** Default assumption—let the data speak (but rarely the best option).
3. **Inverse variance** Weight by $1/\text{Var}(x_k)$ so high-variance variables don't dominate.
4. **Correlation-based** Down-weight variables highly correlated with others (reduce redundancy).

Advanced approaches (part of where my research lies!): entropy-based weighting, feature importance from supervised models, optimization-based methods that maximize cluster validity indices, nonparametric approaches, and more...

A Special Case of Continuous Data

Latitude and longitude look like continuous variables...

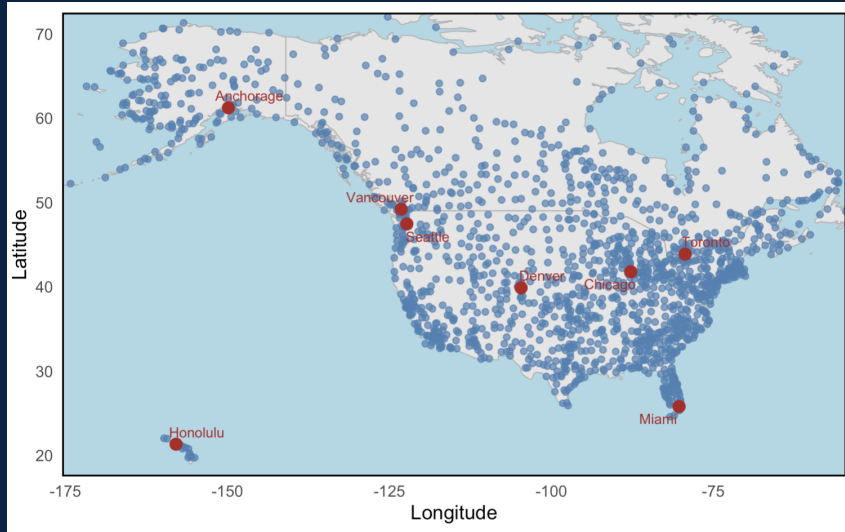
But Euclidean distance on lat/lon is **wrong** (and I tend to never say wrong)!

- At equator: 1° longitude ≈ 111 km
- At 45° latitude: 1° longitude ≈ 78 km
- At 60° latitude: 1° longitude ≈ 55 km

Euclidean treats all degrees as equal distances.

Even displaying points on a 2D map is a distortion (the Earth is a sphere).

North American Airports



Great-Circle Distance on a Sphere

The **Haversine formula** computes the shortest path along Earth's surface

$$a = \sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)$$
$$d = 2R \cdot \text{atan2} \left(\sqrt{a}, \sqrt{1-a} \right)$$

where ϕ = latitude, λ = longitude, R = Earth's radius (6371 km), $\Delta\phi$ is the difference between two coordinates (latitude), and d is the final Haversine distance.

```
haversine <- function(lat1, lon1, lat2, lon2) {  
  R <- 6371  
  dLat <- (lat2-lat1)*pi/180  
  dLon <- (lon2-lon1)*pi/180  
  lat1r <- lat1*pi/180  
  lat2r <- lat2*pi/180  
  a <- sin(dLat/2)^2 + cos(lat1r)*cos(lat2r)*sin(dLon/2)^2  
  c <- 2*atan2(sqrt(a), sqrt(1-a))  
  return(R*c)  
}
```

Haversine vs Euclidean: City Pairs



Vancouver to Seattle:
Haversine: 205 km
Euclidean: 1.95 (raw lat/lon units)
Ratio (Hav/Euc): 104.78 km per unit

Vancouver to Toronto:
Haversine: 3355 km
Euclidean: 44.14 (raw lat/lon units)
Ratio (Hav/Euc): 76.02 km per unit

Miami to Houston:
Haversine: 1549 km
Euclidean: 15.62 (raw lat/lon units)
Ratio (Hav/Euc): 99.17 km per unit

Anchorage to Miami:
Haversine: 6428 km
Euclidean: 78.05 (raw lat/lon units)
Ratio (Hav/Euc): 82.35 km per unit

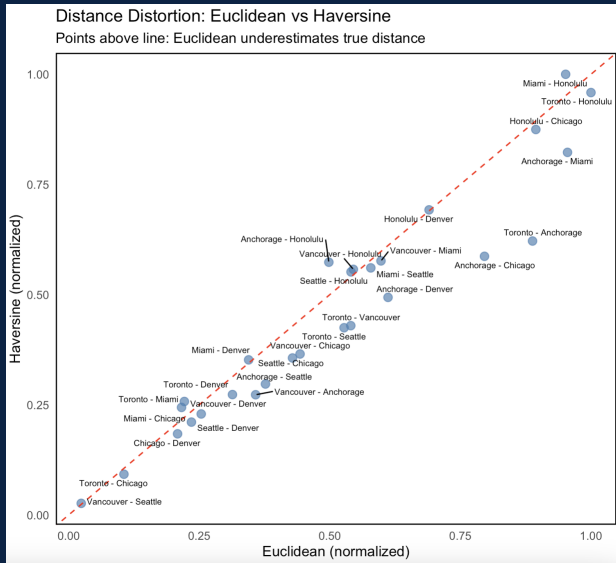
New York to Los Angeles:
Haversine: 3965 km
Euclidean: 45.06 (raw lat/lon units)
Ratio (Hav/Euc): 88.01 km per unit

The ratio of Haversine to Euclidean varies by latitude: if Euclidean and Haversine were comparable, the ratio would be approximately the same throughout.

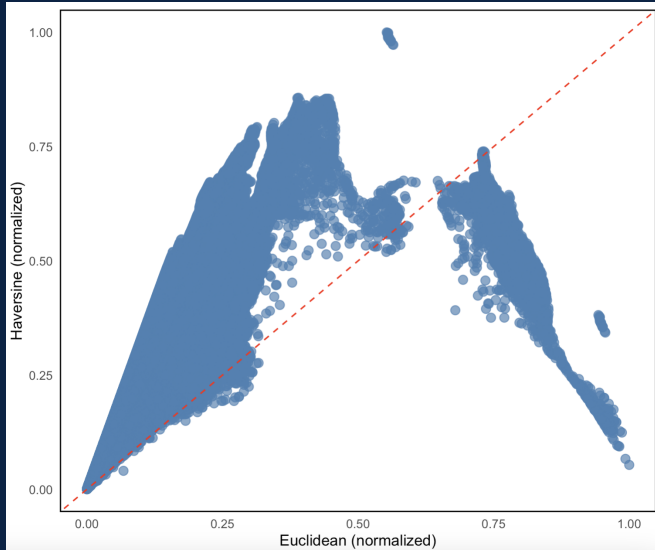
Out of the red points on the map, vertical pairs appear most distorted.

	pair	distortion
	Toronto - Miami	1.1627419
	Anchorage - Honolulu	1.1514259
	Miami - Chicago	1.1294740
	Vancouver - Seattle	1.0961014
	Miami - Honolulu	1.0509516
	Vancouver - Honolulu	1.0225559
	Miami - Denver	1.0214965
	Seattle - Honolulu	1.0202635
	Honolulu - Denver	1.0033233
	Honolulu - Chicago	0.9779684

Quantifying the Distortion



Comparison of All Airports



Summary: Choosing a Distance Measure



Method	Variable Types	Strengths	Limitations
Euclidean (raw)	Continuous	Simple	Scale-dependent
Euclidean (z-score)	Continuous	Scale-invariant	Assumes normality
Euclidean (min-max)	Continuous	Bounded $[0, 1]$	Outlier-sensitive
Euclidean (dummy)	Mixed (encoded)	Handles categories	Loses ordinal info
Gower	Mixed	Native mixed-type	Equal category distance
Podani	Mixed	Better ordinal	More complex
Huang	Mixed	Fast for k -prototypes	Needs tuning
Haversine	Geographic	Accurate on sphere	Only for lat/lon

** Incorporating Haversine into Gower is a simple extension of Gower's distance. **

- Real data is often mixed-type: continuous, ordinal, nominal, binary
- Naive approaches (scaling, dummy coding) have potentially serious limitations
- Gower distance properly handles mixed types with type-specific components
- Alternatives like Podani and Huang offer different trade-offs
- Variable weights incorporate domain knowledge
- Geographic data requires Haversine, not Euclidean distance
- Compare multiple measures to understand sensitivity of results
- Distance choice affects downstream clustering (next workshop!)
- No single “best” measure—depends on data and domain context

What's Next?



Additional Questions?
Book an Appointment!



Next Workshop: Mar 11, 1:00 PM

Mixed-Type Data Clustering

→ Now that we can compute distances, how do we cluster?

Thank You!

Questions?

Workshop Materials:

<https://csc-ubc-okanagan.github.io/workshops/>

Contact:

jesse.ghashti@ubc.ca